

面试八股文-就业冲刺

三. Linux系统编程阶段

面试八股文-就业冲刺

三. Linux系统编程阶段

1. 为什么要用Linux系统
2. 文件总结
 - 2.1 文件描述符
 - 2.2 常用API接口
 - 2.3 文件IO库函数和Linux系统库有什么区别
 - 2.4 Linux文件有什么应用场景
3. 进程总结
 - 3.1 进程概念
 - 3.2 进程状态
 - 3.3 进程创建
 - 3.4 进程终止
 - 3.5 进程控制
 - 3.6 进程调度
 - 3.7 进程标识
 - 3.8 僵尸进程
 - 3.9 孤儿进程
 - 3.10 守护进程
 - 3.11 执行新程序
4. 进程间通信总结和使用场景
 - 4.1 进程间通信有哪些方式
 - 4.1 管道
 - 4.1.1 什么是管道
 - 4.1.2 在进程间通信的时候，什么场景用管道多
 - 4.1.3 示例代码
 - 4.2 信号
 - 4.2.1 什么是信号
 - 4.2.2 在进程间通信的时候，什么场景用信号多
 - 4.2.3 示例代码
 - 4.3 消息队列
 - 4.3.1 什么是消息队列
 - 4.3.2 在进程间通信的时候，什么场景用消息队列多
 - 4.3.3 描述一个场景使用消息队列
 - 4.3.4 消息队列示例代码
 - 4.4 共享内存
 - 4.4.1 什么是共享内存
 - 4.4.2 在进程间通信的时候，什么场景用共享内存多
 - 4.4.3 描述一个场景使用共享内存
 - 4.4.4 示例代码
 - 4.5 信号量
 - 4.5.1 什么是信号量
 - 4.5.2 什么是原子操作
 - 4.5.3 使用信号量的场景举例
 - 4.5.4 示例代码
 - 4.7 套接字
 - 4.7.1 什么是套接字
 - 4.7.2 在进程间通信，什么场景用套接字比较多
 - 4.7.3 场景举例
 - 4.7.4 示例代码

- 5. 线程总结
 - 5.1 什么是线程
 - 5.2 如何创建和终止线程
 - 5.3 线程调度和优先级
 - 5.4 线程调度示例
- 6. 线程同步总结
 - 6.1 什么是线程同步
 - 6.2 线程同步的方法和技术
 - 6.2.1 互斥锁
 - 6.2.2 信号量
 - 6.2.3 互斥锁和信号量区别
 - 6.2.4 死锁
 - 6.2.5 线程条件
 - 6.2.6 线程条件和互斥锁的区别
- 7. 网络编程总结
 - 7.1 Socket
 - 7.2 TCP协议
 - 7.2.1 TCP是什么
 - 7.2.2 三次握手四次挥手-背诵
 - 7.3 UDP协议
 - 7.3.1 UDP是什么-背诵
 - 7.4 Socket的TCP通信代码
 - 7.5 Socket的UDP通信代码
 - 7.6 select机制
 - 7.6.1 面试官会问
 - 7.6.2 select函数说明
 - 7.6.3 select示例代码
 - 7.7 poll机制
 - 7.7.1 什么是poll
 - 7.7.2 示例代码
 - 7.7.3 和select区别
- 8. 第三方库使用总结
 - 8.2 为什么使用第三方库
 - 8.2 使用步骤
 - 8.3 什么是动态库
 - 8.3.1 开发并使用动态库
 - 8.3.2 使用动态库
 - 8.3.3 使用一个现成动态库
 - 8.3.4 第三库一般安装流程
- 9. Linux常用命令汇总

1. 为什么要用Linux系统

首先Linux免费开源，有丰富的硬件资源适配，完整的操作系统支持，开源免费支持个性化定制。

尽管RTOS在实时性和资源受限的环境中有其优势，Linux因其多功能性和灵活性，在更广泛的嵌入式应用中被广泛采用。

目前很多嵌入式应用场景对性能要求很高，比如安卓手机底层就是Linux，或者运行机器人ROS系统，AI方案很多都要依赖于Linux系统，

所以作为嵌入式软件开发工程师，肯定要学习Linux。同时关于操作系统的线程调度，网络通信等内容可以在开发能力上向下兼容RTOS，

有一定的Linux开发经验之后，对嵌入式行业的各种操作系统上手非常快速。

2. 文件总结

Linux文件编程是Linux操作系统下进行文件操作的编程实践，涉及到文件的创建、读取、写入、修改和删除等操作。

以下是一些基本的Linux文件编程概念和操作：

2.1 文件描述符

- 在Linux中，文件和设备都被抽象为文件描述符（file descriptor），这是一个非负整数，用于标识一个打开的文件。

2.2 常用API接口

1. 打开文件：

- 使用 `open()` 函数打开文件，并返回一个文件描述符。

```
int fd = open("filename", O_RDONLY);
```

2. 读取文件：

- 使用 `read()` 函数从文件描述符读取数据。

```
ssize_t bytes_read = read(fd, buffer, sizeof(buffer));
```

3. 写入文件：

- 使用 `write()` 函数将数据写入文件描述符。

```
ssize_t bytes_written = write(fd, buffer, sizeof(buffer));
```

4. 关闭文件：

- 使用 `close()` 函数关闭文件描述符。

```
close(fd);
```

5. 文件定位：

- 使用 `lseek()` 函数在文件中移动读写位置。

```
off_t offset = lseek(fd, 0, SEEK_SET); // 移动到文件开头
```

6. 文件属性：

- 使用 `stat()` 和 `fstat()` 函数获取文件状态信息，如大小、权限、最后修改时间等。

```
struct stat file_stat;  
fstat(fd, &file_stat);
```

7. 文件创建：

- 使用 `creat()` 函数创建新文件，并打开用于写入。

```
int fd = creat("newfile", 0644);
```

8. 文件删除:

- 使用 `unlink()` 函数删除文件。

```
unlink("filename");
```

9. 目录操作:

- 使用 `opendir()`、`readdir()` 和 `closedir()` 函数进行目录操作。

```
DIR *dir = opendir("directory");
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    // Process entry
}
closedir(dir);
```

10. 文件权限和所有权:

- 使用 `chmod()` 和 `chown()` 函数改变文件的权限和所有权。

```
chmod("filename", 0755);
chown("filename", user_id, group_id);
```

11. 文件锁定:

- 使用 `flock()` 函数对文件进行锁定，以实现文件级别的并发控制。

```
flock(fd, LOCK_EX);
```

12. 文件I/O库函数:

- C标准库提供了更高级别的文件I/O函数，如 `fopen()`、`fread()`、`fwrite()`、`fclose()` 等。

这些是Linux文件编程的一些基本操作。在实际编程中，还需要考虑错误处理、文件缓冲、I/O效率和并发访问等问题。

2.3 文件I/O库函数和Linux系统库有什么区别

Linux系统对文件操作有两套不同的API组，

文件I/O库函数（如C标准库中的 `fopen`、`fread`、`fwrite`、`fclose` 等）提供了对文件进行操作的高级抽象，

它们在用户空间中工作，隐藏了底层的系统调用细节，使得文件操作更加方便和安全；

而Linux系统库函数（如 `open`、`read`、`write`、`close` 等）是直接与操作系统内核交互的低级接口，它们提供了更接近硬件层面的文件操作能力，

允许更精细的控制，但需要程序员处理更多的细节，包括文件描述符管理和错误处理等。

简而言之，文件I/O库函数是对Linux系统库函数的封装，提供了更易于使用的API，而系统库函数则提供了更底层、更灵活但更复杂的文件操作能力。

2.4 Linux文件有什么应用场景

Linux文件在应用场景中扮演着核心角色，它们不仅用于存储和检索数据，如文本文件、配置文件、日志文件等，

还作为程序执行的数据源或目标，支持文件系统操作和文件属性管理，实现进程间通信（如通过共享文件或管道），

并作为系统资源管理的基础，如设备文件允许用户空间程序与硬件设备进行交互，

临时文件用于缓存和会话数据存储，从而构成了操作系统中数据处理和交换的基石。

3. 进程总结

学习Linux进程管理是理解操作系统核心概念的关键部分。以下是对Linux进程学习的一个总结：

3.1 进程概念

进程是操作系统进行资源分配和调度的基本单位，是程序执行的实体。

经常问：程序和进程有什么区别？？？

回答：进程是运行起来的程序，操作系统会分配运行内存空间。程序是静态的概念，是存在硬盘上的一段代码，而进程是动态概念，操作系统为其分配资源

3.2 进程状态

进程有多种状态，包括就绪、运行、阻塞和僵尸状态。

3.3 进程创建

通过 `fork()` 创建子进程，它允许父进程创建一个与自己几乎完全相同的进程。

这里对fork调用机制要熟悉，比如根据返回值判断当前是父进程还是子进程，父子进程的内存空间，还有父子进程通过管道来交换数据。

3.4 进程终止

使用 `exit()` 结束进程生命周期，`return` 语句也可以从 `main()` 函数返回退出代码。

面试官问：父进程如何等待或者收集子进程退出的状态？？？？？

回答：在Linux中，父进程可以使用 `wait()` 或 `waitpid()` 系统调用来等待一个或多个子进程结束。这些调用会阻塞父进程的执行，直到指定的子进程结束运行或者超时，具体函数的API需要通过man手册查看

以下是 `wait()` 和 `waitpid()` 的基本用法：

1. `wait()`函数：

```
#include <sys/wait.h>
pid_t wait(int *status);
```

- `wait()` 函数使父进程挂起，直到其任何一个子进程结束或产生一个停止信号。
- 如果调用 `wait()` 时没有子进程已经结束，父进程将被阻塞。
- `status` 参数用于获取子进程的退出状态。

2. waitpid()函数:

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
```

- waitpid() 允许父进程等待特定的子进程。
- pid 参数指定要等待的子进程的PID。
- status 参数用于获取子进程的退出状态。
- options 参数可以是 WNOHANG（非阻塞等待）、WUNTRACED（等待子进程停止）、WCONTINUED（等待子进程继续执行）等，以控制等待的行为。

使用 wait() 或 waitpid() 时，父进程会阻塞直到子进程结束，

此时子进程的状态会被回收，变成僵尸进程（如果父进程没有读取其状态）。

父进程需要读取子进程的退出状态，以避免子进程长时间保持僵尸状态。

如果父进程没有调用 wait() 或 waitpid()，子进程的资源将不会被完全释放，这可能导致资源泄露。

在多进程编程中，正确使用 wait() 或 waitpid() 是非常重要的，它确保了父进程能够妥善处理子进程的结束，并释放相关资源。

3.5 进程控制

kill() 函数用于发送信号给进程，可以用于进程控制和通信。

3.6 进程调度

Linux使用调度算法（如CFS）来决定哪个进程将获得CPU时间。

面试官问：Linux操作系统是如何调度进程的？？？

回答：Linux进程调度核心是通过一个称为调度器的组件来实现的，它负责决定哪个进程应该获得CPU时间以及何时获得。

调度器基于进程的优先级、状态、nice值和调度策略（如时间片轮转、优先级调度等）来选择下一个要运行的进程。

调度器还负责处理进程的创建、睡眠、唤醒和终止等事件，确保系统资源的有效利用和进程间的协调运行。

面试官问：你是如何在应用层代码中干涉或管理进程调度的？？？

回答：在应用层可以通过多种方式影响进程调度，例如通过调整进程的nice值来改变其在队列中的优先级，

使用信号和线程控制来改变进程状态，

或者利用多线程和异步I/O技术来优化程序的响应时间和资源利用率，

从而间接影响操作系统的调度决策，实现对进程执行顺序和时机的控制。

3.7 进程标识

每个进程都有一个唯一的进程ID（PID），用于标识进程。

3.8 僵尸进程

完成执行但未被父进程回收状态的进程，需要通过 `wait()` 系列函数处理。

3.9 孤儿进程

父进程退出后，子进程成为孤儿进程，由 `init` 进程接管。

3.10 守护进程

后台运行，不与任何终端相关联的进程。

3.11 执行新程序

`exec()` 系列函数或者 `system` 函数允许在当前进程中执行新程序，替换当前进程的地址空间。

面试官问：`exec` 和 `system` 有什么区别？

回答：`exec` 函数族（如 `execl`、`execp`、`execv` 等）替换当前进程的映像为一个新的程序，

而不会创建新的进程，原先的进程环境和内存空间被新程序完全覆盖，且 `exec` 调用后，如果成功则不会返回；

而 `system` 函数会创建一个子进程来执行指定的命令，原进程会阻塞直到子进程完成，

简而言之，`exec` 用于在当前进程中替换程序，而 `system` 用于在新进程中运行程序并同步等待其完成。

4. 进程间通信总结和使用场景

4.1 进程间通信有哪些方式

Linux 进程间通信（IPC）主要包括管道（包括无名管道和有名管道 FIFO）、信号、消息队列，共享内存、信号量、套接字，这些方式使得不同进程能够交换信息和同步操作，以实现复杂的并发程序设计。

4.1 管道

4.1.1 什么是管道

管道是 Unix 和类 Unix 操作系统中用于进程间通信的一种机制，

允许一个进程的输出直接连接到另一个进程的输入，实现数据的单向流动，常用于命令行中将多个命令组合起来处理数据流，

或者在程序中创建无名管道以在父子进程间传递信息。

4.1.2 在进程间通信的时候，什么场景用管道多

在进程间通信时，管道多用于以下场景：

首先是在 Shell 命令行中，管道“|”用于将一个命令的输出直接作为另一个命令的输入，

实现多个命令的串联处理，如 `ls -l | grep "txt"`；

其次，管道可用于实现简易的进程池，通过管道父进程可以向子进程派发任务，子进程从管道中读取数据并执行相应操作。

此外，管道也适用于父子或兄弟进程间的数据读写，以及在需要单向数据流的场景中。

因此，管道在需要简单、直接的线性数据传输和命令组合处理的应用场景中使用较多。

4.1.3 示例代码

展示了如何在Linux环境下使用无名管道进行父子进程间的通信

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    // 创建管道
    int fds[2];
    if (pipe(fds) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // 创建子进程
    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) { // 子进程
        // 关闭写端
        close(fds[1]);

        // 从管道读数据
        char buffer[100];
        int num_bytes = read(fds[0], buffer, sizeof(buffer));
        if (num_bytes == -1) {
            perror("read");
            exit(EXIT_FAILURE);
        }
        buffer[num_bytes] = '\0'; // 确保字符串结束
        printf("Child process received: %s\n", buffer);

        // 关闭读端
        close(fds[0]);
        exit(EXIT_SUCCESS);
    } else { // 父进程
        // 关闭读端
        close(fds[0]);

        // 向管道写数据
        const char* message = "Hello, world!";
        write(fds[1], message, sizeof(message));
```



```

        // 关闭写端
        close(fds[1]);

        // 等待子进程结束
        wait(NULL);
    }

    return 0;
}

```

4.2 信号

4.2.1 什么是信号

进程间通信的信号（Signal）是一种软件中断机制，

它允许进程之间或一个进程内部发送和接收特定的信息，以通知接收进程某个事件已经发生，

如用户按键、硬件异常或进程请求等，信号可以被看作是一种快速的、异步的通信方式，

它允许进程对各种事件做出响应，进行相应的处理，如终止进程、暂停执行或忽略信号等。

4.2.2 在进程间通信的时候，什么场景用信号多

在使用进程间通信时，信号（Signal）多用于以下场景：

- 处理异步事件，如用户中断（Ctrl+C）、终止进程、非法内存访问等；
- 通知进程进行状态变更，如挂起（STOP）和继续（CONT）进程；
- 处理作业控制，如终端登录和退出信号；以及在不同进程间传递简单信息，如报警（ALARM）和定时器信号。
- 信号也常用于进程的异常终止，如终止子进程或响应致命错误。

因此，信号在需要快速响应系统或用户生成事件，以及需要异步通知进程状态变更的应用场景中使用较多。

4.2.3 示例代码

下面是一个简单的C语言示例，展示了如何在Linux环境下使用信号进行进程间通信：

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// 信号处理函数
void signal_handler(int sig) {
    printf("Received signal %d\n", sig);
}

int main() {
    // 安装信号处理函数
    signal(SIGINT, signal_handler); // SIGINT 是 Ctrl+C 的信号

    printf("Program is running... Press Ctrl+C to send SIGINT signal.\n");

    // 使程序暂停，等待信号
}

```

```
while(1) {  
    sleep(1);  
}  
  
return 0;  
}
```

这段代码中，我们定义了一个信号处理函数 `signal_handler`，它会在接收到特定信号时被调用。

在 `main` 函数中，我们使用 `signal` 函数将 `SIGINT` 信号（通常由用户按下 `Ctrl+C` 生成）与我们的处理函数关联起来。

然后程序进入一个无限循环，等待用户按下 `Ctrl+C` 发送 `SIGINT` 信号。

当信号发生时，`signal_handler` 函数会被调用，并打印一条消息。程序将继续运行，直到被外部中断或主动退出。

实际上，我们在 `shell` 终端按下 `Ctrl+C` 就是一个进程行为，然后这个信号会传输给上述程序，上述程序收到信号后调用对应处理函数。

4.3 消息队列

4.3.1 什么是消息队列

消息队列是一种进程间通信机制，它允许不同进程将消息发送到一个队列中，其他进程可以从这个队列中读取消息，

这种方式支持进程间的数据交换，允许消息在不同进程间按照先进先出的顺序进行传递，并且可以跨多个系统进行通信。

消息队列可以看作是一个在多个进程之间共享的字节缓冲区，提供了一种从一个进程向另一个进程发送数据的异步方式，特别适用于分布式系统的通信。

4.3.2 在进程间通信的时候，什么场景用消息队列多

消息队列在进程间通信中多用于需要可靠消息传递、数据共享和进程解耦的场景，

例如分布式系统中不同服务之间的通信、多进程系统中跨多个进程的数据交换、任务队列管理、事件通知以及日志记录等。

它特别适用于那些对消息顺序、完整性和持久性有较高要求的应用，

并且当进程可能暂时不可用或需要异步处理消息时，消息队列提供了一个缓冲机制，确保消息不会因为接收者暂时的不可用而丢失。

因此，在需要跨网络通信、大规模分布式应用、或者需要保证消息顺序和可靠性的场景中，消息队列是一种常用的进程间通信方式。

4.3.3 描述一个场景使用消息队列

让我们设想一个智能家居系统中的点灯控制场景，其中消息队列用于协调不同组件之间的通信：

1. 用户操作：

- 用户通过智能家居应用（如手机APP）发出指令，要求打开或关闭某个房间的灯。

2. 消息发送：

- 智能家居应用将用户的指令封装成一条消息，并发送到消息队列中。这条消息包含了动作（开或关）、房间编号和时间戳等信息。

3. 中央控制系统：

- 智能家居的中央控制系统订阅了消息队列，监听来自用户的控制指令。当新消息到达时，中央控制系统解析消息内容，并根据指令更新其内部状态，记录下哪个房间的灯需要被控制。

4. 执行器响应：

- 中央控制系统将控制命令发送给相应的执行器（如智能开关或继电器模块），这些执行器也订阅了来自中央控制系统的消息。执行器接收到消息后，会立即执行开灯或关灯操作。

5. 状态反馈：

- 执行器在完成操作后，可以向中央控制系统发送一个状态更新消息，确认灯的状态已经改变。中央控制系统将这个状态更新再次发送到消息队列中，以便其他系统组件（如能源管理系统）可以据此进行相应的调整。

6. 系统解耦：

- 通过使用消息队列，各个系统组件（用户界面、中央控制系统、执行器）之间的直接依赖被消除，提高了系统的灵活性和可维护性。

7. 可靠性和容错：

- 如果中央控制系统或执行器暂时无法处理消息，消息队列会保留这些消息，直到它们变得可用并能够处理这些消息。这确保了即使在系统部分组件暂时失效的情况下，用户指令也不会丢失。

通过使用消息队列，智能家居系统能够确保用户的点灯指令在各个系统组件之间可靠、高效地传递，并且允许系统以异步的方式响应用户指令，提高了系统的响应速度和用户体验。同时，消息队列的使用还增强了系统的扩展性和容错能力。

4.3.4 消息队列示例代码

在Linux系统中，可以使用系统V消息队列或POSIX消息队列来实现进程间通信。以下是使用POSIX消息队列的一个简单示例代码：

首先，是创建和发送消息的代码（msg_send.c）：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mqueue.h>

int main() {
    mqd_t mq;
    struct mq_attr attr;
    char *message = "Hello, world!";

    // 设置消息队列属性
    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = 80;
    attr.mq_curmsgs = 0;

    // 创建消息队列
    mq = mq_open("/myqueue", O_CREAT | O_WRONLY, 0644, &attr);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        exit(1);
    }

    // 发送消息
```

```

    if (mq_send(mq, message, strlen(message) + 1, 0) == -1) {
        perror("mq_send");
        exit(1);
    }

    printf("Sent: %s\n", message);

    // 关闭消息队列
    mq_close(mq);

    return 0;
}

```

然后，是接收消息的代码（msg_receive.c）：

```

#include <stdio.h>
#include <stdlib.h>
#include <mqueue.h>

int main() {
    mqd_t mq;
    char buffer[80];

    // 打开消息队列
    mq = mq_open("/myqueue", O_RDONLY);
    if (mq == (mqd_t)-1) {
        perror("mq_open");
        exit(1);
    }

    // 接收消息
    if (mq_receive(mq, buffer, 80, NULL) == -1) {
        perror("mq_receive");
        exit(1);
    }

    printf("Received: %s\n", buffer);

    // 关闭消息队列
    mq_close(mq);

    // 删除消息队列
    mq_unlink("/myqueue");

    return 0;
}

```

在这两个程序中，`msg_send.c` 创建了一个名为 `/myqueue` 的消息队列并向其发送了一条消息。`msg_receive.c` 打开同一个消息队列并等待接收消息。接收完毕后，它关闭并删除了消息队列。

为了编译和运行这些程序，需要在终端中执行以下命令：

```
gcc msg_send.c -o msg_send -lrt
gcc msg_receive.c -o msg_receive -lrt
./msg_send
./msg_receive
```

请注意，编译时需要链接实时库（-lrt），因为POSIX消息队列的函数需要这个库。在运行接收程序之前，请确保发送程序已经运行并发送了消息，否则接收程序可能会阻塞等待消息的到来。

4.4 共享内存

4.4.1 什么是共享内存

共享内存是一种进程间通信机制，它允许两个或多个进程共享一个给定的存储区，

这些进程可以直接读写这块内存区域中的数据而无需进行数据拷贝，从而实现高效的数据交换和通信，特别适用于需要频繁交换大量数据的场景。

4.4.2 在进程间通信的时候，什么场景用共享内存多

共享内存存在进程间通信中多用于以下场景：需要频繁交换大量数据的情况，

如图像处理、科学计算和大型数据处理；多个进程需要访问和更新共享数据结构，如数据库的缓存；

以及性能要求高的实时系统，其中通信延迟需要最小化。

共享内存因其高效的数据访问速度和简单的编程模型而被广泛应用于这些场景，尤其是在进程间需要紧密协作，且对性能要求较高的应用中。

4.4.3 描述一个场景使用共享内存

在一个视频处理系统中，假设我们需要将一段高清视频流进行实时的解码和渲染。

这个过程中，视频解码器进程负责将视频流解码成帧数据，而视频渲染器进程则负责将解码后的帧数据显示到屏幕上。

由于视频数据量巨大，如果使用传统的进程间通信方式（如管道或消息队列），每次传递帧数据都需要进行复制，这将导致显著的性能开销和延迟。

在这种情况下，共享内存就显得非常有用。

视频解码器进程可以将解码后的帧数据直接写入到一个共享内存区域，

而视频渲染器进程可以直接从这个共享内存区域读取帧数据进行渲染，无需数据复制。

这样，两个进程共享同一块内存区域，可以极大地提高数据处理的效率，减少延迟，实现流畅的视频播放体验。

使用共享内存的场景优势在于：

1. **减少数据复制**：避免了在不同进程间传递大数据时的复制开销。
2. **提高效率**：进程可以直接读写共享内存，无需额外的数据序列化和反序列化步骤。
3. **降低延迟**：适用于对实时性要求高的应用，如视频处理和游戏开发。
4. **简化编程模型**：相比于复杂的通信机制，共享内存的编程模型更简单直观。

总之，共享内存存在处理大规模数据共享的场景下，如视频拷贝和处理，提供了一种高效、低延迟的进程间通信方式。

4.4.4 示例代码

下面是一个使用共享内存的简单C语言程序示例，它由两个部分组成：一个共享内存的创建者（写者）和一个共享内存的使用者（读者）。

写者程序 (shared_mem_writer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define SHM_NAME "/my_shared_memory"
#define SHM_SIZE 4096

int main() {
    // 创建共享内存对象
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        exit(EXIT_FAILURE);
    }

    // 设置共享内存的大小
    ftruncate(shm_fd, SHM_SIZE);

    // 映射共享内存
    char *ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    // 写入数据到共享内存
    const char *message = "Hello from the writer!";
    strcpy(ptr, message);

    printf("writer wrote: %s\n", ptr);

    // 解除映射
    munmap(ptr, SHM_SIZE);

    // 关闭共享内存对象
    close(shm_fd);

    return 0;
}
```

读者程序 (shared_mem_reader.c)

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>

#define SHM_NAME "/my_shared_memory"
#define SHM_SIZE 4096

int main() {
    // 打开共享内存对象
    int shm_fd = shm_open(SHM_NAME, O_RDONLY, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        exit(EXIT_FAILURE);
    }

    // 映射共享内存
    char *ptr = mmap(NULL, SHM_SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    // 从共享内存读取数据
    printf("Reader read: %s\n", ptr);

    // 解除映射
    munmap(ptr, SHM_SIZE);

    // 关闭共享内存对象
    close(shm_fd);

    // 删除共享内存对象
    shm_unlink(SHM_NAME);

    return 0;
}

```

编译这两个程序：

```

gcc shared_mem_writer.c -o shared_mem_writer -lrt
gcc shared_mem_reader.c -o shared_mem_reader -lrt

```

运行程序：

1. 先运行写者程序：

```
./shared_mem_writer
```

2. 然后运行读者程序：

```
./shared_mem_reader
```

请注意，编译时需要链接实时库（-lrt），因为某些系统调用（如 `mmap`）需要这个库。写者程序创建了一个名为 `/my_shared_memory` 的共享内存对象，写入了一条消息，然后解除映射并关闭共享内存。读者程序打开同一个共享内存对象，读取消息，然后解除映射、关闭共享内存，并删除共享内存对象以释放资源。

4.5 信号量

4.5.1 什么是信号量

信号量是一种同步机制，用于控制多个进程或线程对共享资源的访问，

它是一个整数变量，可以对其执行两种原子操作：

- P操作（等待，如果信号量的值大于0则减1，否则阻塞）
- V操作（信号，使信号量的值加1，如果有进程因此被唤醒则允许它们继续执行）从而实现进程间的互斥与同步。

4.5.2 什么是原子操作

原子操作是指在多线程环境中，一个操作或者一系列操作要么全部执行并且执行过程不会被任何其他线程中断，要么就全部都不执行，

即这些操作要么完全成功，要么在出现任何异常时完全不对系统状态产生影响。

这种操作因其不可分割性而得名“原子”，类似于原子作为物质不可分割的最小单位。

在编程中，原子操作常用于实现锁和同步机制，保证数据的一致性和线程安全。

4.5.3 使用信号量的场景举例

在智能家居系统中，假设我们有一个由多个灯泡组成的照明系统，这些灯泡连接到一个中心控制面板，该面板由一个或多个进程控制。我们可以使用信号量来管理对灯泡的访问，以确保在更新灯泡状态时不会发生冲突。以下是一个具体的“点灯”场景：

1. 灯泡状态更新：

- 假设有多个用户可以通过不同的设备（手机、平板、语音助手等）控制同一组灯泡的颜色和亮度。
- 每个控制请求都会生成一个进程或线程，这些进程需要安全地更新灯泡的状态。

2. 使用信号量：

- 定义一个信号量，初始值为1，表示灯泡的状态信息是互斥的。
- 每个进程在更新灯泡状态之前执行P操作（等待），如果信号量大于0，则将其减1并继续执行。
- 进程更新完灯泡状态后执行V操作（信号），将信号量加1，允许其他等待的进程更新灯泡状态。

3. 场景描述：

- 用户A通过手机APP请求将客厅的灯泡颜色设置为蓝色。
- 同时，用户B通过语音命令请求将同一灯泡的亮度调暗。
- 控制面板接收到这两个请求，并为每个请求创建一个进程。
- 两个进程尝试同时更新灯泡状态，但信号量确保它们不会同时进行。
- 用户A的进程首先获得信号量，更新灯泡颜色为蓝色。
- 用户B的进程等待信号量，直到用户A的进程释放信号量后，继续执行并调整灯泡亮度。

通过使用信号量，我们确保了即使在高并发的情况下，灯泡的状态更新也是线程安全的，避免了颜色和亮度设置之间的冲突，从而保证了照明系统的稳定运行和用户的控制体验。

4.5.4 示例代码

在Linux系统中，可以使用POSIX信号量来实现进程间的同步。以下是使用POSIX信号量的一个简单示例，包括一个生产者和一个消费者进程。

生产者代码 (producer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    sem_t *sem;
    int fd;

    // 创建或打开一个信号量
    fd = sem_open("mysem", O_CREAT, 0644, 1);
    if (fd == SEM_FAILED) {
        perror("sem_open");
        exit(EXIT_FAILURE);
    }

    sem = (sem_t *)fd;

    // 模拟生产过程
    for (int i = 0; i < 5; i++) {
        printf("Producer is producing...\n");
        sleep(1); // 生产耗时
        sem_post(sem); // 释放信号量，表示有产品可消费
    }

    // 关闭信号量
    sem_close(sem);

    // 删除信号量
    sem_unlink("mysem");

    return 0;
}
```

消费者代码 (consumer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
```

```

sem_t *sem;

// 打开信号量
sem = sem_open("mysem", 0);
if (sem == SEM_FAILED) {
    perror("sem_open");
    exit(EXIT_FAILURE);
}

// 模拟消费过程
for (int i = 0; i < 5; i++) {
    sem_wait(sem); // 等待信号量，表示等待产品
    printf("Consumer is consuming...\n");
    sleep(1); // 消费耗时
}

// 关闭信号量
sem_close(sem);

return 0;
}

```

编译这两个程序：

```

gcc producer.c -o producer -lpthread
gcc consumer.c -o consumer -lpthread

```

运行程序：

1. 先运行消费者程序：

```
./consumer
```

2. 然后运行生产者程序：

```
./producer
```

请注意，编译时需要链接pthread库（-lpthread），因为信号量的操作需要线程库的支持。在这个例子中，生产者进程模拟生成产品并释放信号量，而消费者进程等待信号量并模拟消费产品。信号量确保了生产者和消费者之间的同步，避免了消费未生产的产品。

4.7 套接字

本质就是socket网络编程，只是网络在单机上构建

4.7.1 什么是套接字

进程间通信的套接字是指使用套接字（Socket）机制在不同进程之间进行数据通信的方法。

在Unix和类Unix系统中，套接字不仅可以用于网络通信，还可以用于本机进程间的通信。

通过将一个进程的输出通过套接字发送给另一个进程，可以实现进程间的数据交换。

这种方式类似于网络通信，但是数据传输发生在本地主机上，不需要经过网络协议栈。

使用套接字进行进程间通信的优点包括灵活的通信方式（支持面向流或面向数据报的通信）、强大的功能（可以进行加密、压缩等操作）以及跨平台的兼容性。

简而言之，进程间通信的套接字是一种利用网络通信协议栈实现本机或跨网络进程间数据交换的技术。

4.7.2 在进程间通信，什么场景用套接字比较多

进程间通信选择套接字非常合适的场景是在一个需要高可靠性和全双工通信的分布式系统中，特别是当通信双方位于不同主机上，

或者即使在同一主机上也需要通过网络协议栈提供的抽象层来实现复杂的通信模式，如长连接、数据流控制和多种协议（TCP/UDP）支持时，

套接字提供了一种标准化且高效的通信机制。

4.7.3 场景举例

在一个智能家居系统中，假设我们有一个中心控制服务器，负责管理整个家庭的智能设备，包括各种智能灯泡。这个服务器可以通过网络与分布在不同房间的智能灯泡进行通信。以下是套接字在这种“点灯”场景中的应用：

1. 中心控制服务器：

- 服务器上运行一个进程，该进程作为TCP服务器，监听特定端口上的连接请求。
- 服务器使用套接字绑定到一个网络接口和端口上，等待来自智能灯泡的连接或控制命令。

2. 智能灯泡：

- 每个智能灯泡都有一个嵌入式系统，能够通过网络与中心控制服务器通信。
- 当灯泡需要报告状态或接收控制命令时，它们使用套接字建立与服务器的TCP连接。

3. 场景描述：

- 用户通过移动设备上的智能家居应用发送指令，比如“打开客厅的灯”。
- 应用将这个指令通过网络发送到中心控制服务器。
- 服务器接收到指令后，通过已建立的TCP连接将指令发送给客厅的智能灯泡。
- 智能灯泡接收到指令，通过解析数据包中的信息来执行相应的操作，比如打开或关闭灯泡，调整亮度和颜色。

在这个场景中，套接字的使用允许中心控制服务器与各个智能灯泡之间进行可靠的、双向的网络通信。服务器可以实时监控和控制每个灯泡的状态，而灯泡也可以将状态变化反馈给服务器，实现智能家居系统中的集中管理和控制。

4.7.4 示例代码

下面是一个简单的示例，展示了如何使用套接字进行本地进程间通信。这个示例包括两个程序：一个服务器（server.c）和一个客户端（client.c）。

服务器程序（server.c）

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>

#define PORT 8080
```

```

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    const char *message = "Hello from server";

    // 创建套接字
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // 绑定套接字到端口
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // 监听端口
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    // 接受连接
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
        (socklen_t *)&addrlen)) < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    // 读取数据
    read(new_socket, buffer, 1024);
    printf("Message from client: %s\n", buffer);

    // 发送数据
    send(new_socket, message, strlen(message), 0);
    printf("Hello message sent\n");

    // 关闭套接字
    close(new_socket);
    close(server_fd);

    return 0;
}

```

客户端程序 (client.c)

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 8080

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char *message = "Hello from client";
    char buffer[1024] = {0};

    // 创建套接字
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // 将IPv4地址从文本转换为二进制形式
    if(inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }

    // 连接到服务器
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // 发送数据
    send(sock, message, strlen(message), 0);
    printf("Hello message sent\n");

    // 接收数据
    read(sock, buffer, 1024);
    printf("Message from server: %s\n", buffer);

    // 关闭套接字
    close(sock);

    return 0;
}

```

编译这两个程序：

```

gcc server.c -o server
gcc client.c -o client

```

运行程序：

1. 先运行服务器程序：

```
./server
```

2. 然后运行客户端程序：

```
./client
```

这个示例中，服务器程序监听8080端口，等待客户端的连接。客户端程序连接到服务器，并发送一条消息。服务器接收这条消息，然后发送一条响应消息回客户端。这个简单的示例展示了使用套接字进行本地进程间通信的基本流程。

5. 线程总结

5.1 什么是线程

线程是操作系统能够进行运算调度的最小单位，是被系统独立调度和分派的基本单位，比进程更小的能独立运行的基本单位，

线程自身基本上不拥有系统资源，只拥有一点在运行中必不可少的资源，但它可以与同属一个进程的其他线程共享进程所拥有的全部资源。

5.2 如何创建和终止线程

C语言中使用POSIX线程（pthread）

创建线程：

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

// 线程函数原型
void* thread_function(void* arg) {
    // 线程执行的代码
    printf("Thread is running\n");
    return NULL;
}

int main() {
    pthread_t thread_id; // 线程ID
    // 创建线程
    if (pthread_create(&thread_id, NULL, thread_function, NULL) != 0) {
        perror("pthread_create");
        exit(EXIT_FAILURE);
    }

    // 等待线程结束
    pthread_join(thread_id, NULL);

    return 0;
}
```

在这个例子中，`pthread_create` 函数用于创建一个新线程，`thread_function` 是线程执行的函数。`pthread_join` 函数用于等待线程结束。

终止线程：

```
// 在线程函数中退出
void* thread_function(void* arg) {
    // 线程执行的代码
    printf("Thread is exiting\n");
    return NULL; // 返回值，主线程可以通过pthread_join获取
}

// 强制终止线程（不推荐）
pthread_cancel(thread_id);
```

线程可以通过在线程函数中正常返回来结束，或者使用 `pthread_cancel` 函数来请求取消线程。`pthread_cancel` 会向线程发送一个取消请求，线程可以选择忽略或响应这个请求。

5.3 线程调度和优先级

线程调度和优先级是操作系统并发管理的关键组成部分，它们共同决定了线程执行的顺序和时机。以下是线程调度和优先级的基本概念：

1. 线程调度 (Thread Scheduling)：

- 线程调度是操作系统决定哪个线程应该获得CPU时间片（CPU时间的一个小单位）的机制。
- 操作系统使用不同的调度算法（如轮转调度、优先级调度、最短作业优先等）来决定线程的执行顺序。
- 现代操作系统通常采用抢占式调度，允许高优先级的线程中断低优先级的线程。

2. 线程优先级 (Thread Priority)：

- 线程优先级是一个数值，用于指示线程的相对重要性。
- 具有较高优先级的线程更有可能被调度执行。
- 优先级的具体数值和范围取决于操作系统的实现。

3. 静态优先级与动态优先级：

- 静态优先级是在线程创建时设置的，并且在线程的整个生命周期中保持不变。
- 动态优先级可以根据线程的行为和系统策略动态调整。

4. 优先级反转 (Priority Inversion)：

- 优先级反转是一种情况，其中高优先级的线程被低优先级的线程阻塞，而低优先级的线程又在等待中等优先级的线程释放资源。
- 这可能导致高优先级的线程无法及时执行，操作系统通过使用优先级继承等技术来解决这个问题。

5. 实时调度与非实时调度：

- 实时调度保证线程在特定时间内响应和执行，通常用于需要严格时间限制的系统。
- 非实时调度更常见，它不提供严格的时间保证。

6. 操作系统特定的线程调度：

- 不同的操作系统有不同的线程调度策略和优先级设置方法。
- 在Linux上，可以使用 `nice` 值来调整进程的优先级，而线程通常继承其父进程的优先级。

5.4 线程调度示例

在Linux系统中，线程的优先级可以通过 `nice` 值来设置，`nice` 值越小，线程的优先级越高。以下是使用 `nice` 值和 `setpriority` 系统调用来设置和获取线程优先级的示例代码。

设置线程优先级 (setpriority.c)

```
#include <stdio.h>
#include <sys/resource.h>
#include <unistd.h>

int main() {
    // 设置进程（线程）的nice值，数值越小优先级越高
    int nice_value = -5; // 通常nice值的范围是-20到19
    if (setpriority(PRIO_PROCESS, getpid(), nice_value) == -1) {
        perror("setpriority");
        return 1;
    }

    printf("Priority set to %d\n", nice_value);
    pause(); // 等待任意信号，以便观察优先级效果
    return 0;
}
```

获取线程优先级 (getpriority.c)

```
#include <stdio.h>
#include <sys/resource.h>
#include <unistd.h>

int main() {
    // 获取当前进程（线程）的nice值
    int nice_value = getpriority(PRIO_PROCESS, getpid());
    if (nice_value == -1) {
        perror("getpriority");
        return 1;
    }

    printf("Current priority is %d\n", nice_value);
    pause(); // 等待任意信号，以便观察优先级效果
    return 0;
}
```

编译这两个程序：

```
gcc setpriority.c -o setpriority
gcc getpriority.c -o getpriority
```

运行程序：

1. 运行 `setpriority` 程序来设置进程的nice值：

```
./setpriority
```


2. 运行 `getpriority` 程序来获取进程的nice值：

```
./getpriority
```

请注意，需要有适当的权限（通常是root权限）来设置低于一定值的nice值。

`setpriority` 函数用于设置进程或线程的优先级，`getpriority` 函数用于获取当前进程或线程的优先级。

`PRIO_PROCESS` 表示我们正在设置或获取进程的优先级，`getpid()` 用于获取当前进程的ID。

在多线程程序中，通常可以通过对线程的原生句柄操作来设置线程的优先级，但请注意，线程优先级的行为可能因操作系统而异。

在某些系统中，线程优先级可能不会影响线程的调度，因为Linux主要使用完全公平调度（CFS）算法，它更倾向于按比例分配CPU时间。

6. 线程同步总结

6.1 什么是线程同步

线程同步是确保多个线程在访问共享资源时能够保持数据一致性和程序正确性的一种机制。

在多线程程序中，如果多个线程同时访问同一资源，而至少有一个线程在修改该资源，就可能出现竞态条件（race condition），

导致数据错误和不可预测的行为。

线程同步通过协调线程的执行顺序来避免这种情况。

6.2 线程同步的方法和技术

6.2.1 互斥锁

互斥锁是一种同步机制，用于保护共享资源，确保在任何时刻只有一个线程能够访问临界区，

防止多个线程同时对共享资源进行写入操作，从而避免竞态条件和数据不一致的问题，

互斥锁要求线程在进入临界区之前获取锁，

在离开临界区后释放锁，其他试图获取同一互斥锁的线程将被阻塞直到锁可用。

以下是使用C语言和POSIX线程库（pthread）实现互斥锁的示例代码。

这个示例包括两个部分：一个是使用互斥锁的线程函数，另一个是主函数，它创建多个线程并使用互斥锁来同步对共享资源的访问。

互斥锁示例代码

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// 定义一个互斥锁
pthread_mutex_t lock;

// 共享资源
int shared_data = 0;
```

```

// 线程函数，增加共享数据
void* increment(void* arg) {
    pthread_mutex_lock(&lock); // 获取互斥锁

    // 临界区开始
    shared_data++;
    printf("Shared data incremented to: %d\n", shared_data);
    // 临界区结束

    pthread_mutex_unlock(&lock); // 释放互斥锁

    return NULL;
}

int main() {
    pthread_t threads[4];
    int i;

    // 初始化互斥锁
    pthread_mutex_init(&lock, NULL);

    // 创建4个线程，每个线程都增加共享数据
    for (i = 0; i < 4; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    // 等待所有线程完成
    for (i = 0; i < 4; i++) {
        pthread_join(threads[i], NULL);
    }

    // 销毁互斥锁
    pthread_mutex_destroy(&lock);

    return 0;
}

```

编译这个程序：

```
gcc -o mutex_example mutex_example.c -lpthread
```

运行程序：

```
./mutex_example
```

这个示例中，我们定义了一个全局的互斥锁 `lock` 和一个共享资源 `shared_data`。`increment` 函数是线程执行的函数，它在修改共享资源之前获取互斥锁，在修改后释放互斥锁。主函数中，我们初始化互斥锁，创建四个线程，每个线程都调用 `increment` 函数来增加共享资源的值，最后等待所有线程完成并销毁互斥锁。

6.2.2 信号量

线程同步信号量是一种用于控制多个线程对共享资源访问的同步机制，

它通过一个计数器来跟踪资源的使用状态，允许多个线程等待信号量、减少计数器值，并在资源可用时增加计数器值，

从而协调线程间的执行顺序，确保共享资源在任一时刻只被一个或有限数量的线程访问。

以下是使用POSIX信号量进行线程同步的示例代码。这个示例包括两个部分：

一个是使用信号量同步的线程函数，另一个是主函数，它创建两个线程并使用信号量来同步对共享资源的访问。

信号量示例代码

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

// 定义一个信号量
sem_t sem;

// 共享资源
int shared_data = 0;

// 线程函数，增加共享数据
void* increment(void* arg) {
    // 等待信号量
    sem_wait(&sem);

    // 临界区开始
    shared_data++;
    printf("Thread %ld: Shared data incremented to: %d\n", pthread_self(),
shared_data);
    // 临界区结束

    // 释放信号量
    sem_post(&sem);

    return NULL;
}

int main() {
    pthread_t threads[2];
    int i;

    // 初始化信号量，初始值为1
    sem_init(&sem, 0, 1);

    // 创建2个线程，每个线程都增加共享数据
    for (i = 0; i < 2; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    // 等待所有线程完成
```

```

    for (i = 0; i < 2; i++) {
        pthread_join(threads[i], NULL);
    }

    // 销毁信号量
    sem_destroy(&sem);

    return 0;
}

```

编译这个程序：

```
gcc -o semaphore_example semaphore_example.c -lpthread
```

运行程序：

```
./semaphore_example
```

这个示例中，我们定义了一个全局的信号量 `sem` 和一个共享资源 `shared_data`。

`increment` 函数是线程执行的函数，它在修改共享资源之前等待信号量（`sem_wait`），在修改后释放信号量（`sem_post`）。主函数中，我们初始化信号量，创建两个线程，每个线程都调用 `increment` 函数来增加共享资源的值，最后等待所有线程完成并销毁信号量。

6.2.3 互斥锁和信号量区别

互斥锁和信号量都是用于线程同步的机制，但互斥锁主要用于保护临界区，确保同一时间只有一个线程可以访问共享资源，

而信号量则更通用，可以用来控制对共享资源的访问数量，允许多个线程同时访问资源，也可用于线程间的条件同步和信号传递。

简而言之，互斥锁是信号量的一种特殊情况，其计数器始终为1。

6.2.4 死锁

死锁是指在多线程或多进程环境中，两个或多个进程在执行过程中因争夺资源而造成的一种僵局，

此时每个进程都在等待其他进程释放资源，但因为没有任何进程能够向前推进，导致所有涉及的进程都无法继续执行，除非外部干预。

如何避免死锁-面试会问

避免线程死锁通常需要遵循一些编程准则和策略，

如确保线程请求资源时遵循固定的顺序，避免资源的循环等待；

确保线程在持有资源时不会请求更多的资源，或者在请求新资源前释放已持有的资源；

使用超时机制来避免无限期等待；

以及实现资源的有序释放和请求，

这样可以减少或消除线程间相互等待资源的情况，从而有效预防死锁的发生。

死锁的错误示例代码：

下面是一个简单的死锁示例代码，使用C语言和POSIX线程库（pthread）来展示两个线程相互等待对方持有的锁，从而导致死锁的情况。

```
#include <pthread.h>
#include <stdio.h>

// 定义两个互斥锁
pthread_mutex_t mutex1;
pthread_mutex_t mutex2;

// 线程1函数
void* thread1(void* arg) {
    // 获取mutex1
    pthread_mutex_lock(&mutex1);
    printf("Thread 1: Locked mutex1\n");

    // 等待mutex2（被线程2持有）
    pthread_mutex_lock(&mutex2);
    printf("Thread 1: Locked mutex2\n");

    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);

    return NULL;
}

// 线程2函数
void* thread2(void* arg) {
    // 获取mutex2
    pthread_mutex_lock(&mutex2);
    printf("Thread 2: Locked mutex2\n");

    // 等待mutex1（被线程1持有）
    pthread_mutex_lock(&mutex1);
    printf("Thread 2: Locked mutex1\n");

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    // 初始化互斥锁
    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    // 创建两个线程
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    // 等待线程结束
    pthread_join(t1, NULL);
```

```
pthread_join(t2, NULL);

// 销毁互斥锁
pthread_mutex_destroy(&mutex1);
pthread_mutex_destroy(&mutex2);

return 0;
}
```

编译这个程序：

```
gcc deadlock_example.c -o deadlock_example -lpthread
```

运行程序：

```
./deadlock_example
```

这个示例中，我们定义了两个互斥锁 `mutex1` 和 `mutex2`。线程1首先锁定 `mutex1`，然后尝试锁定 `mutex2`。线程2首先锁定 `mutex2`，然后尝试锁定 `mutex1`。由于两个线程都在等待对方释放锁，因此程序进入死锁状态，没有任何线程能够继续执行。

6.2.5 线程条件

线程条件控制是一种同步机制，

它允许线程在某些条件不满足时挂起（阻塞）自己的执行，

并在条件变为真时被唤醒。

这种机制通常与互斥锁（mutex）结合使用，以保护共享资源和条件状态。

线程条件控制的关键组成部分包括：

1. 条件变量（Condition Variable）：

- 条件变量是一种特殊的同步对象，线程可以在上面等待或被唤醒。
- 它通常与互斥锁一起使用，以确保在检查条件、挂起线程和唤醒线程时数据的一致性。

2. 等待条件：

- 当线程需要等待某个条件成立时，它会在条件变量上执行等待（wait）操作。
- 在等待期间，线程会释放与之关联的互斥锁，以便其他线程可以访问共享资源。

3. 通知和唤醒：

- 当条件变量关联的条件变为真时，其他线程可以通知（signal）或广播（broadcast）等待该条件的所有线程。
- 被通知的线程将重新尝试获取互斥锁，并重新检查条件是否满足。

4. 避免竞态条件：

- 条件变量的使用可以避免竞态条件，因为它确保了线程在检查条件和访问共享资源时持有互斥锁。

5. 使用场景：

- 线程条件控制适用于生产者-消费者问题、读者-写者问题等需要协调多个线程对共享资源访问的场景。

线程条件控制的目的是让线程在没有必要执行时节省CPU资源，同时确保当它们需要执行时能够及时响应。正确使用条件变量可以提高程序的效率和响应性，并减少忙等待（busy-waiting）带来的资源浪费。

示例代码：

6.2.6 线程条件和互斥锁的区别

线程条件控制（条件变量）与互斥锁（Mutex）都是用于线程同步的机制，但它们的目的是用法有所不同：

总结来说，互斥锁主要用于保护共享资源，而条件变量用于控制线程的执行时机。在实际编程中，它们经常结合使用，以实现复杂的线程同步和协调。

1. 目的不同：

- **互斥锁**：主要用于保护共享资源，确保在任何时刻只有一个线程可以访问临界区。
- **条件变量**：用于线程间的协调，允许线程在某个条件不满足时挂起（阻塞），并在条件满足时被唤醒。

2. 用法不同：

- **互斥锁**：线程在进入临界区前获取锁，在离开临界区后释放锁。
- **条件变量**：线程在检查条件不满足时，通过条件变量挂起，并释放与之关联的互斥锁；在条件满足时，被其他线程唤醒，并重新获取互斥锁。

3. 作用范围：

- **互斥锁**：用于保护对共享资源的访问，防止多个线程同时访问。
- **条件变量**：用于控制线程执行的时机，使线程在特定条件下等待或继续执行。

4. 唤醒机制：

- **互斥锁**：没有内置的唤醒机制，线程需要在获取锁后主动检查条件。
- **条件变量**：提供了等待和唤醒操作，线程可以在条件变量上等待，直到被其他线程通过信号唤醒。

5. 使用场景：

- **互斥锁**：适用于需要严格互斥访问的场景，如数据保护、临界区控制等。
- **条件变量**：适用于需要基于条件同步的场景，如生产者-消费者问题、读者-写者问题等。

下面是一个使用POSIX线程库（pthread）中的条件变量和互斥锁来实现生产者-消费者问题的示例代码。这个示例包括两个线程，一个生产者线程和一个消费者线程，它们共享一个缓冲区来存储数据。

生产者-消费者示例代码

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

// 共享缓冲区
int buffer[BUFFER_SIZE];
int count = 0;

// 互斥锁和条件变量
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```

pthread_cond_t can_produce = PTHREAD_COND_INITIALIZER;
pthread_cond_t can_consume = PTHREAD_COND_INITIALIZER;

// 生产者函数
void* producer(void* arg) {
    int item;
    for (int i = 0; i < 20; i++) {
        item = i; // 生产一个项目
        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE) {
            pthread_cond_wait(&can_produce, &mutex); // 缓冲区满，等待
        }
        buffer[count++] = item; // 将项目放入缓冲区
        printf("Produced: %d\n", item);
        pthread_cond_signal(&can_consume); // 通知消费者可以消费
        pthread_mutex_unlock(&mutex);
        sleep(1); // 模拟生产时间
    }
    return NULL;
}

// 消费者函数
void* consumer(void* arg) {
    for (int i = 0; i < 20; i++) {
        pthread_mutex_lock(&mutex);
        while (count == 0) {
            pthread_cond_wait(&can_consume, &mutex); // 缓冲区空，等待
        }
        int item = buffer[--count]; // 从缓冲区取出项目
        printf("Consumed: %d\n", item);
        pthread_cond_signal(&can_produce); // 通知生产者可以生产
        pthread_mutex_unlock(&mutex);
        sleep(1); // 模拟消费时间
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    // 创建生产者和消费者线程
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // 等待线程结束
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // 销毁互斥锁和条件变量
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&can_produce);
    pthread_cond_destroy(&can_consume);

    return 0;
}

```


编译这个程序：

```
gcc -o prod_cons prod_cons.c -lpthread
```

运行程序：

```
./prod_cons
```

这个示例中，我们定义了一个固定大小的共享缓冲区 `buffer` 和一个计数器 `count` 来跟踪缓冲区中的项目数。生产者线程生成项目并将其放入缓冲区，消费者线程从缓冲区取出项目。使用互斥锁 `mutex` 来保护对共享缓冲区的访问，使用条件变量 `can_produce` 和 `can_consume` 来同步生产者和消费者线程。

7. 网络编程总结

7.1 Socket

套接字（Socket）网络编程是一种允许不同主机上的进程进行双向通信的技术，

它基于TCP/IP协议栈，支持面向连接（如TCP）和无连接（如UDP）的通信模式，

使得开发者可以创建客户端和服务端应用程序，实现数据的发送、接收以及错误处理等功能，广泛应用于构建网络服务和分布式系统。

7.2 TCP协议

7.2.1 TCP是什么

TCP（传输控制协议）通信是一种面向连接的、可靠的、基于字节流的通信协议，（背诵）

它在IP网络中为应用程序提供了一种可靠的数据传输服务，确保数据包按顺序、无差错、不重复地从源传送到目的地，

通过三次握手建立连接、数据传输过程中的流量控制和拥塞控制以及四次挥手终止连接的机制，保证了通信的可靠性和稳定性。

7.2.2 三次握手四次挥手-背诵

三次握手（Three-way Handshake）是TCP协议中建立连接的过程，

客户端和服务端通过交换SYN（同步序列编号）和ACK（确认应答）标志的三个包来确认彼此的接收和发送能力，从而建立一个可靠的连接；

四次挥手（Four-way Wavehand）是TCP协议中终止连接的过程，任一端通过发送FIN（结束）标志来请求关闭连接，对方回应ACK表示收到，

并在数据传输结束后发送自己的FIN标志，最后接收方确认并关闭连接，完成连接的释放。

7.3 UDP协议

7.3.1 UDP是什么-背诵

UDP（用户数据报协议）是一种无连接、不可靠的传输层通信协议，

它提供了一种简单的方式来发送封装的IP数据报，但不保证数据的顺序、完整性或重复性，适用于对实时性要求高且可以容忍一定数据丢失的应用场景。

7.4 Socket的TCP通信代码

下面是一个简单的TCP通信示例，包括一个服务器端和一个客户端。服务器端将监听特定端口，等待客户端的连接请求，然后接收客户端发送的消息并回显给客户端。

服务器端代码 (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 创建套接字文件描述符
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // 绑定套接字到端口8080
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // 监听端口
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    // 接受客户端连接
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
        (socklen_t *)&addrlen)) < 0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    // 读取客户端发送的数据
    read(new_socket, buffer, BUFFER_SIZE);
    printf("Message from client: %s\n", buffer);

    // 发送应答给客户端
```

```

const char *msg = "Hi, client. I have received your message\n";
send(new_socket, msg, strlen(msg), 0);
printf("Message sent to client\n");

// 关闭套接字
close(new_socket);
close(server_fd);

return 0;
}

```

客户端代码 (client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char *message = "Hello from client";
    char buffer[1024] = {0};

    // 创建套接字文件描述符
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // 将IPv4地址从文本转换为二进制形式
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address / Address not supported \n");
        return -1;
    }

    // 连接到服务器
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }

    // 发送数据
    send(sock, message, strlen(message), 0);
    printf("Hello message sent\n");

    // 接收数据
    read(sock, buffer, 1024);
    printf("Message from server: %s\n", buffer);
}

```

```
// 关闭套接字
close(sock);

return 0;
}
```

编译这两个程序：

```
gcc server.c -o server
gcc client.c -o client
```

运行程序：

1. 先运行服务器程序：

```
./server
```

2. 然后运行客户端程序：

```
./client
```

这个示例中，服务器程序监听8080端口，等待客户端的连接请求。客户端程序连接到服务器，并发送一条消息。服务器接收这条消息，然后发送一条响应消息回客户端。这个简单的示例展示了使用套接字进行TCP通信的基本流程。

7.5 Socket的UDP通信代码

下面是使用UDP协议的简单Socket通信示例，包括一个服务器端和一个客户端。服务器端将监听特定端口，等待客户端发送的数据，然后回显给客户端。

服务器端代码 (udp_server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    char buffer[BUFFER_SIZE] = {0};
    socklen_t addrlen = sizeof(address);
    int bytes_received;

    // 创建套接字文件描述符
    if ((server_fd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }
}
```

```

}

// 绑定套接字到端口8080
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("bind failed");
    exit(EXIT_FAILURE);
}

printf("UDP server up and listening on port %d\n", PORT);

// 接收客户端数据
bytes_received = recvfrom(server_fd, buffer, BUFFER_SIZE, 0, (struct sockaddr
*)&address, &addrlen);
if (bytes_received < 0) {
    perror("recvfrom failed");
    exit(EXIT_FAILURE);
}

printf("Message from client: %s\n", buffer);

// 发送应答给客户端
sendto(server_fd, buffer, strlen(buffer), 0, (struct sockaddr *)&address,
sizeof(address));
printf("Message sent to client\n");

// 关闭套接字
close(server_fd);

return 0;
}

```

客户端代码 (udp_client.c)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define SERVER_IP "127.0.0.1"

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char *message = "Hello from client";
    char buffer[1024] = {0};

    // 创建套接字文件描述符
    if ((sock = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }
}

```

```

}

serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);

// 将IPv4地址从文本转换为二进制形式
if (inet_pton(AF_INET, SERVER_IP, &serv_addr.sin_addr) <= 0) {
    printf("\nInvalid address / Address not supported \n");
    return -1;
}

// 发送数据
sendto(sock, message, strlen(message), 0, (struct sockaddr *)&serv_addr,
sizeof(serv_addr));
printf("Hello message sent\n");

// 接收数据
recvfrom(sock, buffer, 1024, 0, NULL, NULL);
printf("Message from server: %s\n", buffer);

// 关闭套接字
close(sock);

return 0;
}

```

编译这两个程序：

```

gcc udp_server.c -o udp_server
gcc udp_client.c -o udp_client

```

运行程序：

1. 先运行服务器程序：

```
./udp_server
```

2. 然后运行客户端程序：

```
./udp_client
```

这个示例中，服务器程序监听8080端口，等待客户端的UDP数据包。客户端程序发送一条消息到服务器，服务器接收这条消息，然后发送一条响应消息回客户端。这个简单的示例展示了使用套接字进行UDP通信的基本流程。请注意，UDP是不可靠的，不保证数据包的顺序或送达。

7.6 select机制

7.6.1 面试官会问

面试官问：如何监控套接字的状态？

回答：在socket编程中，select机制允许程序同时监控多个套接字的状态，包括是否有数据可读、是否可以写入数据或是否发生错误，通过调用select函数，

程序可以等待一个或多个套接字变得可用，然后执行相应的读写操作，

这使得单个线程能够有效地管理多个网络连接，提高了程序的并发处理能力和响应性。

7.6.2 select函数说明

在C语言中，`select` 函数的原型定义在 `<sys/select.h>` 头文件中，其原型如下：

```
int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct
timeval *timeout);
```

参数说明：

- `nfds`：参数指定了 `readfds`、`writefds` 和 `exceptfds` 三个集合中文件描述符的最大值加1。也就是说，如果你的文件描述符集合中最大的文件描述符编号是 `m`，那么 `nfds` 应该设置为 `m+1`。
- `readfds`：指向 `fd_set` 结构的指针，用于指定想要检查哪些文件描述符是否有数据可读。
- `writefds`：指向 `fd_set` 结构的指针，用于指定想要检查哪些文件描述符是否可以写入数据。
- `exceptfds`：指向 `fd_set` 结构的指针，用于指定想要检查哪些文件描述符是否有异常条件。
- `timeout`：指向 `timeval` 结构的指针，用于指定 `select` 函数的超时时间。如果设置为 `NULL`，`select` 将一直阻塞直到至少有一个文件描述符准备好。

返回值：

- 如果有一个或多个文件描述符准备好了，`select` 返回这些文件描述符的数量。
- 如果发生错误，返回-1。
- 如果超时，返回0。

`fd_set` 是一个打开文件描述符集合，`FD_SET` 宏用于将一个文件描述符添加到集合中，`FD_CLR` 宏用于从集合中移除一个文件描述符，`FD_ISSET` 宏用于检查某个文件描述符是否在集合中。

`timeval` 结构定义如下：

```
struct timeval {
    long tv_sec;      /* 秒 */
    long tv_usec;     /* 微秒 */
};
```

这个结构用于指定超时的时间，其中 `tv_sec` 表示秒数，`tv_usec` 表示微秒数。

7.6.3 select示例代码

以下是使用 `select` 在 C 语言中进行 socket 编程的一个简单示例：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/time.h>
#include <sys/types.h>
```

```
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    fd_set readfds; // 文件描述符集合

    // 创建套接字文件描述符
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // 绑定套接字到端口
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // 监听套接字
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Listening on port %d...\n", PORT);

    // 循环等待客户端连接
    while (1) {
        // 清空文件描述符集合
        FD_ZERO(&readfds);

        // 添加监听套接字到集合中
        FD_SET(server_fd, &readfds);

        // 使用 select 检查套接字状态
        if (select(server_fd + 1, &readfds, NULL, NULL, NULL) < 0) {
            perror("select error");
        }

        // 检查是否有新的连接
        if (FD_ISSET(server_fd, &readfds)) {
            if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
            (socklen_t*)&addrlen)) < 0) {
                perror("accept");
                exit(EXIT_FAILURE);
            }
        }
    }
}
```



```

        printf("Connection accepted\n");

        // 读取数据
        read(new_socket, buffer, BUFFER_SIZE);
        printf("Data received: %s\n", buffer);

        // 发送响应
        const char *msg = "Data received successfully";
        send(new_socket, msg, strlen(msg), 0);
        printf("Message sent\n");

        // 关闭连接
        close(new_socket);
    }
}

return 0;
}

```

这个示例代码创建了一个服务器，它监听在8080端口上。当有新的客户端连接时，服务器会接受连接，并读取客户端发送的数据，然后发送一个确认消息给客户端，并关闭连接。

请注意，这个示例代码是阻塞式的，并且在实际应用中可能需要更多的错误处理和功能，比如非阻塞套接字、多线程或事件驱动模型等。此外，`select` 函数的超时参数在这个示例中被设置为 `NULL`，这意味着 `select` 会一直阻塞直到有文件描述符准备好。在实际应用中，你可能需要设置一个超时时间来避免无限期的阻塞。

7.7 poll机制

7.7.1 什么是poll

`poll` 是一个系统调用，用于监视多个文件描述符（包括套接字），

以确定哪些文件描述符已经准备好进行 I/O 操作（如读取或写入）。

与 `select` 类似，`poll` 可以同时检查多个文件描述符，但它使用不同的数据结构和接口。

在 Linux 系统中，`poll` 函数的原型定义在 `<poll.h>` 头文件中，如下所示：

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout);
```

参数说明：

- `fds`：指向 `pollfd` 结构数组的指针，每个 `pollfd` 结构包含需要检查的文件描述符和相关的事件。
- `nfds`：`fds` 数组的大小。
- `timeout`：等待时间，单位为毫秒。如果设置为 0，则 `poll` 立即返回；如果设置为 -1，则 `poll` 会阻塞直到某个文件描述符就绪。

返回值：

- 成功时返回就绪的文件描述符数量。
- 如果发生错误，返回 -1。

`pollfd` 结构定义如下：

```

struct pollfd {
    int    fd;           /* 文件描述符 */
    short  events;       /* 需要监视的事件类型 */
    short  revents;      /* 发生事件的返回值 */
};

```

- `fd`：需要检查的文件描述符。
- `events`：需要监视的事件类型，可以是以下值的组合：
 - `POLLIN`：文件描述符可读。
 - `POLLOUT`：文件描述符可写。
 - `POLLPRI`：有紧急数据可读。
 - `POLLERR`：发生错误。
 - `POLLHUP`：挂断。
 - `POLLNVAL`：文件描述符无效。
- `revents`：实际发生的事件，由 `poll` 函数填充。

7.7.2 示例代码

以下是一个简单的使用 `poll` 的示例，用于监视一个套接字是否可读：

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <poll.h>

#define PORT 8080

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    struct pollfd fds;

    // 创建套接字
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // 绑定套接字
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
    }
}

```

```

    exit(EXIT_FAILURE);
}

// 监听套接字
if (listen(server_fd, 3) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

printf("Listening on port %d...\n", PORT);

// 设置 pollfd 结构
fds.fd = server_fd;
fds.events = POLLIN;

// 使用 poll 检查套接字
while (1) {
    if (poll(&fds, 1, -1) == -1) {
        perror("poll");
        exit(EXIT_FAILURE);
    }

    if (fds.revents & POLLIN) {
        if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen)) < 0) {
            perror("accept");
            exit(EXIT_FAILURE);
        }

        read(new_socket, buffer, sizeof(buffer));
        printf("Data received: %s\n", buffer);

        close(new_socket);
    }
}

return 0;
}

```

这个示例创建了一个服务器，使用 `poll` 来检查是否有新的连接请求。如果检测到 `POLLIN` 事件，它将接受连接并读取数据。这个示例展示了 `poll` 的基本用法，但在实际应用中，你可能需要更复杂的错误处理和多文件描述符的监视。

7.7.3 和select区别

`poll` 和 `select` 都是用于检查多个文件描述符（如套接字）是否准备好进行 I/O 操作的系统调用，但它们在内部实现和使用上有所不同。

`select` 使用三个独立的集合来跟踪读、写和异常条件，

而 `poll` 使用单个 `pollfd` 结构数组来跟踪所有事件。

`poll` 没有 `select` 的文件描述符数量限制，因此在处理大量文件描述符时通常更高效。

此外，`poll` 的 `timeout` 参数以毫秒为单位，

而 `select` 的 `timeout` 参数以秒和微秒为单位。

总的来说, `poll` 提供了更灵活的接口,

但在现代 Linux 系统中, `epoll` (一种更高效的 I/O 事件通知机制) 通常是首选, 因为它可以更有效地处理大量并发连接。

8. 第三方库使用总结

8.2 为什么使用第三方库

使用第三方库的原因多种多样, 以下是一些常见的理由:

1. **节省时间**: 第三方库通常提供了现成的代码和功能, 开发者可以直接使用, 而不需要从头开始编写所有代码, 这样可以大大节省开发时间。
2. **提高效率**: 第三方库通常由经验丰富的开发者或团队开发和维护, 它们可能经过了优化, 比自己编写的代码更加高效。
3. **减少错误**: 由于第三方库被广泛使用和测试, 它们往往更加稳定和可靠, 使用这些库可以减少自己代码中的错误。
4. **利用专业知识**: 第三方库的开发者可能在特定领域拥有深厚的专业知识, 使用这些库可以让开发者利用这些专业知识, 而不需要自己成为该领域的专家。
5. **跨平台兼容性**: 许多第三方库提供了跨平台的支持, 使得开发者可以编写一次代码, 然后在多个操作系统上运行, 无需为每个平台单独编写代码。
6. **社区支持**: 流行的第三方库通常有一个活跃的社区, 开发者可以从社区中获得帮助, 包括bug修复、新功能请求和最佳实践。
7. **持续更新和维护**: 第三方库的维护者会定期更新库, 修复bug, 添加新功能, 这使得开发者可以持续受益于最新的技术进步。
8. **标准化**: 第三方库可以帮助标准化开发流程和接口, 使得代码更加一致, 易于理解和维护。
9. **安全性**: 许多第三方库, 特别是那些专注于安全的库, 会不断更新以应对新的安全威胁, 使用这些库可以提高应用程序的安全性。
10. **集成**: 现在很多类似openCV图像处理, 讯飞语音处理等接口存在, 开发者要学会这项技能。

总之, 使用第三方库可以让开发者站在巨人的肩膀上, 利用已有的资源和知识, 更快、更有效地开发出高质量的软件产品。

8.2 使用步骤

在Linux系统中使用第三方库通常涉及以下几个步骤:

1. **指定头文件和库文件**: 在使用第三方库时, 需要提供库的头文件和库文件给编译器。如果库没有安装在系统的标准路径下, 需要通过编译选项指定库文件和头文件的位置。
2. **编译时链接**: 在编译程序时, 需要告诉编译器链接哪个库。对于静态库, 使用 `-L` 选项指定库文件路径, 使用 `-l` 选项指定库名 (不包括前缀 `lib` 和后缀 `.a`)。对于动态库, 同样需要指定库文件路径和库名, 但动态库的 `.so` 文件需要放在系统默认的库路径下, 或者通过设置环境变量 `LD_LIBRARY_PATH` 来指定库文件的搜索路径。
3. **动态库的安装**: 动态库通常需要安装到系统的库目录下, 如 `/lib` 或 `/usr/lib`。可以通过 `sudo cp` 命令将动态库文件复制到这些目录下, 以便系统能够找到并加载它们。

4. **设置环境变量**：如果不想将动态库文件复制到系统库目录，可以通过设置 `LD_LIBRARY_PATH` 环境变量来指定动态库的搜索路径。这可以通过在shell中执行 `export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/path/to/library` 来实现，其中 `/path/to/library` 是动态库文件的路径。
5. **动态加载库**：在程序运行时，可以使用 `dlopen`、`dlsym` 和 `dlclose` 等函数动态加载和卸载库，这些函数定义在 `dlfcn.h` 头文件中。
6. **编译选项**：在编译时，对于动态库，需要使用 `-fPIC` 选项来生成位置无关代码，使用 `-shared` 选项来生成共享库。

通过这些步骤，你可以在Linux系统中有效地使用第三方库，无论是静态库还是动态库。

8.3 什么是动态库

Linux动态库是一种在程序运行时才被加载到内存中的共享库文件，通常以 `.so` (Shared Object) 为文件扩展名。

它们允许多个程序共享同一份库代码，节省内存并提高效率；同时，动态库可以在不重新编译依赖它的程序的情况下进行更新，便于功能扩展和错误修复。

8.3.1 开发并使用动态库

在Linux系统中生成一个动态库（共享库）通常涉及以下几个步骤：

1. **编写库的代码**：首先，你需要编写包含你想要共享的函数和资源的代码。这些代码将被编译成动态库的一部分。
2. **编译代码为动态库**：使用编译器（如gcc）将代码编译成动态库。你需要使用 `-shared` 标志来告诉编译器你正在创建一个共享库，并且通常使用 `-fPIC` 标志来生成位置无关代码（Position Independent Code），这对于动态库是必需的。
3. **链接库**：在编译过程中，你可能需要链接其他库文件，使用 `-l` 标志来指定。

下面是一个简单的示例，展示如何生成一个动态库：

假设我们有一个名为 `libmath` 的库，它包含一个简单的函数 `add`，用于计算两个整数的和。

libmath.c:

```
#include "libmath.h"

int add(int a, int b) {
    return a + b;
}
```

libmath.h:

```
#ifndef LIBMATH_H
#define LIBMATH_H

int add(int, int);

#endif
```

编译和生成动态库：

```
gcc -shared -fPIC -o libmath.so libmath.c
```

这条命令将 `libmath.c` 编译并链接成一个名为 `libmath.so` 的动态库。

4. **安装动态库**：生成动态库后，你需要将其放置在系统的标准库目录下（如 `/usr/lib`），或者将其放在你的应用程序的特定目录下，并确保该目录包含在 `LD_LIBRARY_PATH` 环境变量中，以便程序能够找到并加载它。
5. **更新共享库缓存**：在某些系统上，你可能需要更新共享库缓存，以便系统能够识别新安装的库。这可以通过运行 `ldconfig` 命令来完成。

8.3.2 使用动态库

要使用刚才创建的 `libmath.so` 动态库，你需要在你的程序中进行几个步骤：

1. **包含库的头文件**：在你的程序中包含 `libmath.h` 头文件，这样你才能访问库中定义的函数。
2. **链接动态库**：在编译你的程序时，你需要告诉编译器链接到 `libmath.so` 库。通常，你需要使用 `-l` 选项来指定库的名称（不包括前缀 `lib` 和后缀 `.so`）。
3. **动态加载库（可选）**：如果你想要动态加载和卸载库，你可以使用 `dlopen`、`dlsym` 和 `dlclose` 函数。否则，你可以在编译时静态链接库。

以下是如何在你的程序中使用 `libmath` 库的示例：

main.c:

```
#include <stdio.h>
#include "libmath.h"

int main() {
    int result = add(3, 4);
    printf("The result of add(3, 4) is: %d\n", result);
    return 0;
}
```

编译并链接你的程序：

```
gcc -o myprogram main.c -L. -lmath
```

这条命令告诉编译器在当前目录（`-L.`）查找库文件，并链接到 `libmath.so` 库（`-lmath`）。

如果你想要动态加载库，可以使用以下代码：主要是 `dlopen` 这一些列API

dynamic_load.c:

```
#include <stdio.h>
#include <dlfcn.h>

int (*add)(int, int);

int main() {
    void *handle;
    char *error;

    // 打开动态库
```

```

handle = dlopen("./libmath.so", RTLD_LAZY);
if (!handle) {
    fprintf(stderr, "%s\n", dlerror());
    return 1;
}

// 清空之前的错误
dlerror();

// 从动态库中获取符号（函数）
*(void **>(&add) = dlsym(handle, "add");
if ((error = dlerror()) != NULL) {
    fprintf(stderr, "%s\n", error);
    dlclose(handle);
    return 1;
}

// 使用函数
int result = add(3, 4);
printf("The result of add(3, 4) is: %d\n", result);

// 关闭动态库
dlclose(handle);
return 0;
}

```

编译并运行动态加载程序：

```

gcc -o dynamic_load dynamic_load.c -ldl
./dynamic_load

```

这条命令编译 `dynamic_load.c` 并链接到 `libdl` 库，这个库提供了 `dlopen`、`dlsym` 和 `dlclose` 等函数。然后，你可以运行生成的 `dynamic_load` 程序，它将动态加载 `libmath.so` 库，调用 `add` 函数，并打印结果。

请确保 `libmath.so` 库文件在程序运行时是可访问的，可以通过将其放在标准库目录下，或者设置 `LD_LIBRARY_PATH` 环境变量来实现。

8.3.3 使用一个现成动态库

使用openssl的aes加密算法

使用OpenSSL库中的AES加密算法进行数据加密和解密，可以通过直接使用AES算法提供的API或者使用EVP框架来实现。以下是两种方法的简要说明和示例代码：

直接使用AES算法提供的API

这种方法直接调用OpenSSL库中的AES函数来进行加密和解密操作。以下是使用AES算法进行ECB模式加密和解密的示例代码：

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <openssl/aes.h>

int main(void) {

```

```

char userkey[AES_BLOCK_SIZE];
unsigned char *data = malloc(AES_BLOCK_SIZE*3);
unsigned char *encrypt = malloc(AES_BLOCK_SIZE*3 + 4);
unsigned char *plain = malloc(AES_BLOCK_SIZE*3);
AES_KEY key;

// 设置密钥
memset(userkey, 'k', AES_BLOCK_SIZE);
memset(data, 'p', AES_BLOCK_SIZE*3);

// 设置加密key及密钥长度
AES_set_encrypt_key(userkey, AES_BLOCK_SIZE*8, &key);

int len = 0;
// 循环加密，每次只能加密AES_BLOCK_SIZE长度的数据
while(len < AES_BLOCK_SIZE*3) {
    AES_encrypt(data+len, encrypt+len, &key);
    len += AES_BLOCK_SIZE;
}

// 设置解密key及密钥长度
AES_set_decrypt_key(userkey, AES_BLOCK_SIZE*8, &key);
len = 0;
// 循环解密
while(len < AES_BLOCK_SIZE*3) {
    AES_decrypt(encrypt+len, plain+len, &key);
    len += AES_BLOCK_SIZE;
}

// 解密后与原数据是否一致
if(!memcmp(plain, data, AES_BLOCK_SIZE*3)){
    printf("test success\n");
}else{
    printf("test failed\n");
}

// 打印加密结果
printf("encrypt: ");
for(int i = 0; i < AES_BLOCK_SIZE*3 + 4; i++){
    printf("%.2x ", encrypt[i]);
    if((i+1) % 32 == 0){
        printf("\n");
    }
}
printf("\n");

free(data);
free(encrypt);
free(plain);
return 0;
}

```

使用EVP框架

EVP框架提供了一个更高层次的抽象，可以更方便地处理多种加密算法。以下是使用EVP框架进行AES加密和解密的示例代码：


```

#include <stdio.h>
#include <stdlib.h>
#include <openssl/evp.h>
#include <openssl/aes.h>

void aes_encrypt(unsigned char *key, unsigned char *iv, unsigned char *plaintext,
int plaintext_len, unsigned char *ciphertext) {
    EVP_CIPHER_CTX *ctx;

    int len;
    int ciphertext_len;

    // 创建和初始化加密上下文
    if(!(ctx = EVP_CIPHER_CTX_new())) {
        printf("Error creating EVP_CIPHER_CTX\n");
        exit(EXIT_FAILURE);
    }

    // 初始化加密操作
    if(1 != EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, key, iv)) {
        printf("Error initializing EVP_EncryptInit_ex\n");
        exit(EXIT_FAILURE);
    }

    // 对数据进行加密
    if(1 != EVP_EncryptUpdate(ctx, ciphertext, &len, plaintext, plaintext_len)) {
        printf("Error with EVP_EncryptUpdate\n");
        exit(EXIT_FAILURE);
    }
    ciphertext_len = len;

    // 结束加密操作
    if(1 != EVP_EncryptFinal_ex(ctx, ciphertext + len, &len)) {
        printf("Error with EVP_EncryptFinal_ex\n");
        exit(EXIT_FAILURE);
    }
    ciphertext_len += len;

    // 清理加密上下文
    EVP_CIPHER_CTX_free(ctx);
}

void aes_decrypt(unsigned char *key, unsigned char *iv, unsigned char
*ciphertext, int ciphertext_len, unsigned char *plaintext) {
    EVP_CIPHER_CTX *ctx;

    int len;
    int plaintext_len;

    // 创建和初始化解密上下文
    if(!(ctx = EVP_CIPHER_CTX_new())) {
        printf("Error creating EVP_CIPHER_CTX\n");
        exit(EXIT_FAILURE);
    }

```

```

// 初始化解密操作
if(1 != EVP_DecryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, key, iv)) {
    printf("Error initializing EVP_DecryptInit_ex\n");
    exit(EXIT_FAILURE);
}

// 对数据进行解密
if(1 != EVP_DecryptUpdate(ctx, plaintext, &len, ciphertext, ciphertext_len))
{
    printf("Error with EVP_DecryptUpdate\n");
    exit(EXIT_FAILURE);
}
plaintext_len = len;

// 结束解密操作
if(1 != EVP_DecryptFinal_ex(ctx, plaintext + len, &len)) {
    printf("Error with EVP_DecryptFinal_ex\n");
    exit(EXIT_FAILURE);
}
plaintext_len += len;

// 清理解密上下文
EVP_CIPHER_CTX_free(ctx);
}

int main(void) {
    unsigned char key[AES_BLOCK_SIZE] = {'k', 'e', 'y', '\0'};
    unsigned char iv[AES_BLOCK_SIZE] = {'i', 'v', '\0'};
    unsigned char plaintext[] = "This is a plaintext message.";
    unsigned char ciphertext[128];
    unsigned char decryptedtext[128];

    // 加密
    aes_encrypt(key, iv, plaintext, strlen((char *)plaintext), ciphertext);

    // 解密
    aes_decrypt(key, iv, ciphertext, sizeof(ciphertext), decryptedtext);

    printf("Plaintext: %s\n", plaintext);
    printf("Ciphertext: ");
    for(int i = 0; i < sizeof(ciphertext); i++)
        printf("%02x ", ciphertext[i]);
    printf("\n");
    printf("Decryptedtext: %s\n", decryptedtext);

    return 0;
}

```

以上示例展示了如何使用OpenSSL库中的AES算法进行加密和解密操作。在实际应用中，你可能需要根据具体需求选择合适的加密模式（如ECB、CBC等）和密钥长度（如128位、192位或256位）。

8.3.4 第三库一般安装流程

安装类似OpenSSL这样的第三方库通常涉及以下步骤：

1. **下载源码**：从官方网站或提供的链接下载库的源码包。例如，可以从GitHub上下载OpenSSL的源码。
2. **配置环境**：安装必要的依赖项，如编译器、构建工具等。对于OpenSSL，可能需要安装nasm和perl等工具。
3. **配置安装路径**：使用配置脚本设置安装路径。例如，使用 `./config --prefix=/path/to/installation` 指定安装目录。
4. **编译库**：使用 `make` 命令编译源码。例如，运行 `make` 来编译OpenSSL库。
5. **安装库**：编译完成后，使用 `make install` 命令安装库到指定路径。
6. **配置项目**：在你的项目中，配置编译器以包含库的头文件和链接库文件。这通常涉及到设置编译器的包含路径（-I）和库路径（-L），以及链接时需要的库（-l）。
7. **环境变量**：如果需要，更新系统的环境变量，如 `LD_LIBRARY_PATH`（Linux）或 `PATH`（Windows），以包含库文件的路径。
8. **测试**：安装完成后，进行测试以确保库安装成功并且可以正常使用。

对于Windows系统，如果使用Visual Studio，还需要在项目属性中配置包含目录和库目录，以及添加依赖项。对于Linux系统，通常需要解压安装包，进入解压后的目录，然后按照上述步骤进行配置、编译和安装。

这些步骤提供了一个基本的框架，具体细节可能会根据库的不同和操作系统的不同而有所变化。

9.Linux常用命令汇总

在Linux系统中，有许多常用的命令用于执行各种任务。以下是一些最基础和常用的Linux命令：

1. 文件和目录操作

- `ls`：列出目录内容。
- `cd`：改变当前目录。
- `pwd`：显示当前目录。
- `mkdir`：创建新目录。
- `rmdir`：删除空目录。
- `touch`：创建空文件或更新现有文件的时间戳。
- `cp`：复制文件或目录。
- `mv`：移动或重命名文件或目录。
- `rm`：删除文件或目录。
- `cat`：查看文件内容。
- `more`：分页查看文件内容。
- `less`：分页查看文件内容，支持前后翻页。
- `head`：查看文件的前几行。
- `tail`：查看文件的最后几行，常用 `-f` 选项实时跟踪文件新增内容。
- `grep`：搜索文件内容，支持正则表达式。
- `find`：在目录树中查找文件。

- `diff`：比较文件差异。
- `tar`：打包和压缩文件。

2. 系统信息和状态

- `uname`：显示系统信息。
- `whoami`：显示当前用户名。
- `uptime`：显示系统运行时间和平均负载。
- `top`：实时显示系统进程信息。
- `ps`：显示当前进程状态。
- `free`：显示内存使用情况。
- `df`：显示磁盘空间使用情况。
- `du`：显示目录或文件的磁盘使用情况。
- `lsdf`：列出打开的文件。
- `netstat`：显示网络状态。
- `ifconfig`（或 `ip`）：配置和显示网络接口。

3. 用户和权限管理

- `useradd`：创建新用户。
- `userdel`：删除用户。
- `usermod`：修改用户属性。
- `passwd`：更改用户密码。
- `groupadd`：创建新组。
- `groupdel`：删除组。
- `chown`：更改文件所有者。
- `chgrp`：更改文件组。
- `chmod`：更改文件权限。
- `umask`：设置默认文件权限。

4. 包管理

- `apt-get`（Debian/Ubuntu）：软件包管理工具。
- `yum`（Red Hat/CentOS）：软件包管理工具。
- `dnf`（Fedora）：新一代软件包管理工具。
- `zypper`（openSUSE）：软件包管理工具。
- `pacman`（Arch Linux）：软件包管理工具。

5. 网络操作

- `ping`：测试网络连接。
- `ssh`：安全远程登录。
- `scp`：安全复制文件。
- `wget`：下载文件。
- `curl`：传输数据。

6. 文本处理

- `echo`：输出文本。
- `sed`：流编辑器，用于文本替换。
- `awk`：模式扫描和处理语言。
- `sort`：排序文本文件。
- `uniq`：去除重复行。

7. 系统监控和日志

- `dmesg`：显示内核消息。
- `syslog`：系统日志。
- `last`：显示用户登录日志。
- `tail`：查看文件末尾内容，常用于日志文件。

8. 磁盘和文件系统管理

- `fdisk`：磁盘分区表操作。
- `mkfs`：创建文件系统。
- `mount`：挂载文件系统。
- `umount`：卸载文件系统。
- `fsck`：检查文件系统。